

《南山下》

游戏系统策划案

文化叙事型轻度种田游戏 | 2D 像素风格 | Unity 引擎

担任角色：系统策划 | 独立开发

2026 年 6 月

一、项目概述

《南山下》是一款以"文化内容机制化"为设计策略的 2D 像素风格轻度种田游戏。游戏以修复一部失落的民间农书——《南山农书》为叙事主线，将农耕文化、器物知识与古典诗词嵌入任务条件与资源交付流程之中。玩家扮演一位回到南山脚下老宅的年轻人，通过探索村庄、种植作物、与 NPC 对话交付物品，逐步拼凑出农书中失落的知识，在约 10 分钟的完整体验中感知农耕文化的内在价值。

1.1 项目定位

在与《星露谷物语》和《桃源深处有人家》两款代表性竞品的对比中，《南山下》找到了自己的差异化位置。与《星露谷物语》相比，本作不追求复杂的经济模拟与社交系统，而是围绕一条明确的文化叙事主线展开，玩法更轻量、主题更聚焦。与《桃源深处有人家》相比，本作同样关注传统文化表达，但设计重心不在"看"而在"做"——文化内容进入任务条件和资源流转，而非停留在画面和文本层面。

1.2 目标玩家与传达体验

目标玩家为对传统文化有一定兴趣的 18-30 岁青年群体，尤其是独立游戏和像素风格的爱好者。游戏通过轻度种田的交互节奏，让玩家在操作中感受到文化知识 with 劳动经验的紧密关联。结局文字"谨以此作，献给那些从未被记录的劳作者"是整个体验的情感落点，将个人操作行为升华为对农耕文明传承者的致敬。

1.3 技术规格

开发平台：Unity (Editor 运行模式与独立构建包)。目标平台：PC (Windows)，分辨率

1920×1080。美术风格：2D 像素 (Aseprite 绘制)，色调以赭石、青绿和藤黄为主，强调乡土质朴感。总代码量约 4000 行，三个 NPC，四条任务线，七个存档维度。

二、设计方法论：文化内容机制化

本项目的核心设计策略可概括为"文化内容机制化"——即将传统文化知识从游戏的"美学层" (Aesthetics) 下沉至"机制层" (Mechanics) , 使文化内容成为玩家操作的一部分而非附加说明。

这一策略源于对大量文化类游戏在"表达"与"体验"之间断裂的观察: 许多游戏将文化元素停留在视觉包装和文本说明层面, 玩家"看到"了文化, 却未在操作中"感知"到文化。

在《南山下》中, 这一策略的落地体现在以下设计选择上: 游戏不设任何"文化知识"阅读界面或百科弹窗。农耕知识(如小麦的生长周期、石磨的用途)成为任务完成的条件本身——玩家必须亲手种植指定作物、理解作物的种类与用途、将特定粮食交付给特定 NPC, 才能推进叙事。在此过程中, 文化知识不是"被告知"的, 而是在操作的必由之路上"被经历"的。

| 2.1 MDA 框架中的设计位移

MDA (Mechanics-Dynamics-Aesthetics) 框架将游戏设计分为三个层次。在传统文化类游戏中, 文化内容的分布通常呈"倒金字塔": 大量文化元素集中在美学层(视觉风格、背景文本), 少量渗透至动态层(玩家行为模式), 几乎不触及机制层(核心规则)。本设计的核心思路是实现文化内容从美学层向机制层的系统位移, 使三个层次形成均衡分布。

MDA 层次	传统文化游戏	《南山下》设计
Mechanics (机制层)	种植=数值产出 任务=条件检查	种植行为=文化学习行为 任务条件=农耕知识验证
Dynamics (动态层)	玩家追求效率最大化 线性任务链推进	等待作物生长=感受自然节律 探索中偶遇 NPC=沉浸式叙事
Aesthetics (美学层)	文化=视觉主题 知识=阅读文本	文化=可操作的前提条件 知识=资源流转的内在规则

上表展示了《南山下》在 MDA 三个层次上与传统文化游戏的设计差异。核心区别在于: 文化内容不再是附加于规则之上的"包装", 而是规则本身的组成部分。

三、系统架构设计

《南山下》的系统架构以"面向接口、事件驱动、统一序列化"为三项基本原则。以下分别阐述四个核心系统的设计思路与技术决策。

| 3.1 NPC 交互系统：IInteractable 统一接口

三个 NPC（谷伯、李婆婆、沈砚）和农田对象（CropTile）均实现同一接口 IInteractable，玩家按 E 键交互时，系统根据最近的可交互对象类型自动判断执行对话还是农田操作。这一设计避免了硬编码的类型判断，使系统具备良好的可扩展性——新增可交互对象只需实现该接口，无需修改底层交互逻辑。

任务逻辑嵌入每个 NPC 的交互脚本中，不设独立的全局任务系统或任务面板。这种分散式任务架构在小型项目中带来了显著优势：任务状态与 NPC 状态天然绑定，不需要额外的任务 ID 映射层；存档时每个 NPC 的进度（hasMetPlayer、phase1Complete）自然成为 SaveData 的字段，逻辑清晰。

以谷伯为例，其交互脚本包含三个阶段：首次见面（记录 hasMetPlayer）、第一阶段任务（检测背包是否有残缺卷轴 ID=3，满足则消耗并标记 phase1Complete）、第二阶段任务（检测背包是否同时持有小麦 ID=4 和水稻 ID=5，满足则消耗并发放完整农书 ID=8）。任务阶段不可逆，完成后存档记录永久生效。

| 3.2 种植系统：三模块协作架构

种植系统由三个模块协作完成：CropTile（土地对象，实现 IInteractable，管理单块土地的翻土/种植/生长/收获状态机）、CropPanel（弹出式作物选择面板，展示可选作物类型，触发种植指令）、GameTimeManager（游戏时间管理器，负责时间累积与 OnDayPassed 静态事件触发）。

关键设计决策在于时间系统与业务逻辑的解耦。GameTimeManager 不直接调用任何作物的生长方法——它仅在 Update 中累计时间，达到阈值时触发 OnDayPassed 静态事件。CropTile 通过订阅该事件驱动自身状态机，新增响应时间变化的对象只需实现订阅即可，时间管理器无需任何修改。这种事件驱动架构是 MDA 框架中“机制层”良好设计的体现：核心规则（时间流逝）与具体表现（作物生长）之间没有耦合。

作物与土地采用不同 SortingLayer 管理渲染层级，两个 SpriteRenderer 分别控制，使得作物成熟前后的视觉变化与土地的交互检测互相独立。

| 3.3 存档系统：统一数据结构 + 协程加载

存档系统采用统一的 SaveData 数据结构，通过 JsonUtility 序列化为本地 JSON 文件，零外部依赖。

SaveData 覆盖七个存储维度：玩家位置（Vector3 坐标 + 地图边界名称）、背包状态（物品 ID + 槽位索引）、快捷栏状态、作物状态（世界坐标 + CropState 枚举 + 作物类型 + 生长计时器）、宝箱状态（ChestID + 是否已开启）、NPC 进度（NPC 名称 + 是否见面 + 第一阶段是否完成）、游戏天数。

一个重要的技术决策是 SaveController 的加载时序。初始设计中 Start 方法直接调用 LoadGame，在场景跳转时出现初始化时序问题（场景对象尚未完成 Awake，存档数据已覆盖）。解决方案是将 Start 改为 IEnumerator 协程，首帧 yield return null 等待所有对象完成初始化后再执行 LoadGame。这一决策体现了对 Unity 生命周期时序的深入理解。

...

四、核心机制深度分析

4.1 核心玩法循环

游戏的核心循环为：探索 → 种植 → 收获 → 交付 → 叙事推进 → 内容解锁。整个流程由七个主线节点构成，顺序不可跳过，但玩家可在等待作物生长的 2 游戏日间隙自由探索地图。这种"强制推进+自由探索"的双重节奏设计，兼顾了叙事紧凑性与玩家自主感。

规则层面对玩家行为的约束包括：物品掉落后 1.5 秒冷却时间（防止连点误拾）、背包满时无法拾取新物品（强制资源管理）、NPC 任务必须同时满足全部所需物品（防止部分交付的歧义）、拾取提示弹窗有最大显示数量限制（控制信息密度）。这些约束规则的设立目的是引导而非限制——每一条规则都对应一种可能破坏体验的边界情况。

4.2 叙事结构与机制融合

游戏的叙事通过四个 NPC 任务线推进，每个任务的条件设定都有其文化逻辑：残缺卷轴的交付象征"知识的继承"——玩家必须先获得先祖遗物，才能开始修复农书；小麦和水稻的种植要求象征"耕作的实践"——农书的内容必须通过亲自种植来"唤醒"；三种粮食换取石磨象征"劳动的工具化"——生产物

转化为生产工具，形成闭环；完整农书交给沈砚（游历文人）象征"知识的传播"——个人劳作成果汇入更大的文化脉络。

值得注意的是，这些文化隐喻并非通过对话文本"说明"出来，而是通过任务条件的设定"实现"出来。玩家在操作过程中自然经历了"继承→实践→工具化→传播"的完整叙事弧线，文化意义嵌入了机制的骨骼之中。

4.3 三幕式空间叙事

游戏场景分为三个功能分区，通过 MapTransation 系统无缝切换，分别对应叙事的三幕：村庄（T1 上方）对应"启程"——玩家在此接受任务、获得遗物、遇见引导者谷伯；种植区（F2）对应"劳作"——玩家在此亲手实践农耕，体验时间流逝与作物生长；游历之地（T1 下方）对应"完成"——玩家在此遇见文人沈砚，交付完整农书，触发结局。

空间设计上，沈砚被放置在 T1 下方的孤立位置，这一设计选择呼应其"旁观者"的叙事身份——他不在村庄之中，也不参与耕作，而是作为一个外部观察者完成叙事收束。这种空间-身份的一致性设计，体现了关卡设计与叙事设计的协同思考。

五、技术方案与设计反思

5.1 关键代码架构

项目采用以下五项核心架构设计：

IIteractable 接口：面向接口编程，实现 NPC 与场景对象的统一交互入口，消除硬编码类型判断，使系统天然具备扩展性。

静态事件驱动：GameTimeManager.OnDayPassed 作为时间系统与业务模块之间的通信桥梁，所有需要响应时间变化的对象通过事件订阅实现松散耦合。

双层 SpriteRenderer：作物与土地使用不同 SortingLayer，视觉表现与交互检测的解耦使种植系统的状态机可以专注于逻辑层。

协程式加载时序：利用 Unity 的协程生命周期管理存档加载时机，解决了场景初始化与数据覆盖之间的竞态问题。

JsonUtility 零依赖序列化：不引入第三方序列化库，利用 Unity 内置的 JsonUtility 实现统一的 SaveData 保存与恢复，降低项目复杂度与维护成本。

5.2 UI 系统设计

UI 系统包含五个层次：主菜单界面（开始/读档/退出）、游戏 HUD（快捷栏 + 拾取提示）、菜单面板（信息/背包/地图/设置四选项卡）、对话界面（头像 + 名称 + 逐字显示文本）、作物选择面板（弹出式按钮列表）。界面之间的焦点管理通过 Tab 键和 Esc 键统一控制，对话界面的文本推进采用"按 E 立即显示完整文本，再按 E 进入下一句"的双阶段交互模式，兼顾阅读节奏与操作效率。

5.3 音效与转场设计

音效系统采用 BGMManager（跨场景持久化）与 SoundEffectManager（即时音效播放）的双层架构，音量设置通过 PlayerPrefs 写入本地持久化。转场动画分为三种类型：开场前言（白色文字在黑色背景上逐段淡入淡出）、结局动画（黑色遮罩渐入 + 结局文字逐行显示）、场景切换（无加载画面，通过 MapTransation 自动切换地图边界与相机范围）。所有动画使用 UnscaledDeltaTime，确保在 Time.timeScale 为 0 时也能正常播放。

5.4 设计反思与迭代方向

在完成本项目的系统设计后，有以下三点核心反思。第一，"分散式任务系统"在小型项目中表现出色，但若 NPC 数量增加至 10 个以上，任务间的依赖关系和状态管理将变得难以维护——此时应考虑引入轻量级任务管理器作为统一调度层。第二，种植系统的作物类型目前为硬编码枚举，若未来扩展至数十种作物，应重构为 ScriptableObject 驱动的数据配置模式，实现策划可配置的作物属性表。第三，存档系统目前为单一存档位，若引入多存档位需增加 SaveSlot 管理器与存档元数据（时间戳、进度摘要）的展示逻辑。

这些反思方向的共同指向是：当前设计在"小而美"的规模下达到了良好的内聚性和可维护性，但其扩展路径需要从"单一场景下的紧致设计"逐步过渡到"多场景下的模块化架构"。这一认知本身即是本项目的核心设计收获。

...

— 全文完 —

Unity | 2D Pixel Art | ~4000 行代码 | 独立开发